

What you gain, and when training fails

The first half is what a framework gives you beyond the gradients: better optimisers, sensible starting weights, and a GPU. The second half is the practical skill that matters more — a network that runs without errors and learns nothing, and how to read the evidence until you find the one line responsible.

Unit 3 · Libraries

80 minutes

Lab 13 · four broken scripts

Compressible: Part 1 can move to reading

What you gain

Four signatures

Live diagnosis

Lab 13

Before Friday

PART 1 · 30 MINUTES · COMPRESSIBLE TO READING

What the frameworks give you

Autograd you met on Friday. Three more.

1

Optimisers: from plain SGD to Adam

Your update moves every parameter by the same multiple of its gradient. That has two weaknesses: on a narrow valley the walk zig-zags across it, and one learning rate must suit every parameter at once — the problem from Lab 6's two-dimensional extension.

Optimiser	The idea	Update, in words
SGD	step down the current slope	$\theta \leftarrow \theta - \eta \cdot g$
Momentum	keep a running velocity, so consistent directions speed up and zig-zags cancel	$v \leftarrow 0.9 \cdot v + g; \theta \leftarrow \theta - \eta \cdot v$
Adam	momentum, plus a per-parameter step size scaled by how large that parameter's gradients usually are	$\theta \leftarrow \theta - \eta \cdot m^{\wedge} / (\sqrt{v^{\wedge}} + \epsilon)$

Adam is the default in most frameworks because it tolerates a badly chosen learning rate and badly scaled parameters. It is not magic: on a small, well-conditioned problem a carefully tuned plain SGD often does as well or better. Its value is robustness when you cannot tune everything.

Initialisation

Starting weights matter more than they look. All zeros makes every hidden neuron compute the same thing forever — the symmetry never breaks. Weights too large push sigmoids into their flat ends, where the slope is almost zero. Frameworks scale the random starting weights to each layer's width: **Glorot** initialisation for sigmoid and tanh, **He** initialisation for ReLU.

The GPU

Training is dominated by matrix multiplication, and a GPU performs thousands of multiply-adds in parallel. For the 64–16–1 network the CPU is faster, because copying data to the GPU costs more than the arithmetic. For session 23's image network the GPU wins by a wide margin. `model.to("cuda")` is the whole change in PyTorch — once the data is moved too.

Reading a loss curve like evidence

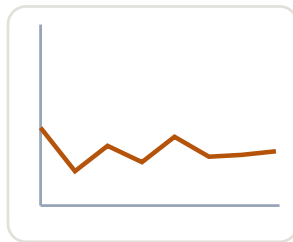
Four signatures cover most failures you will meet this year.

2

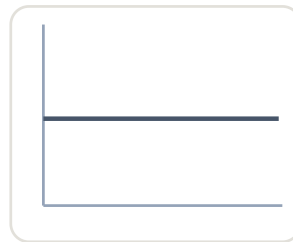
FOUR LOSS CURVES FROM LAB 13 — SKETCHED, NOT TO SCALE



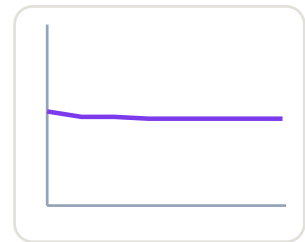
jumps and explodes



stalls and stutters



flat at 0.693



falls a little, no skill

Signature	Usual cause	First thing to print
Loss jumps, grows, or becomes <code>nan</code>	learning rate far too large	the loss for the first five steps
Loss falls then stutters; accuracy well below what it should be	inputs on very different or very large scales	<code>X.min()</code> , <code>X.max()</code> , <code>X.std(axis=0)</code>
Loss flat at exactly 0.693 from the first epoch	no gradient reaches the weights — dead or saturated units	the share of hidden activations above zero
Loss hardly moves; test accuracy near the baseline	labels do not match inputs	a few images next to their labels

Four broken scripts, diagnosed at the front

Each runs without an error. Each has one wrong line.

3

The healthy reference — Lab 11's network, 30 epochs — ends with a loss of 0.056 and 96.2% test accuracy. Keep those numbers on the board.

- 1 **The learning rate.** $\eta = 50$. First-epoch loss 14.0; after thirty epochs still 7.8; test accuracy 53%. The fix is one number: $\eta = 1.0$. Lesson: huge losses that bounce mean the step is overshooting the valley, exactly as on the x^2 curve in session 9.
- 2 **The scale.** Raw pixels, 0 to 16, with the learning rate tuned for 0 to 1. The loss drops to 0.26 then jumps back to 0.9; test accuracy 82%. The step on each weight is sixteen times larger than intended. The fix is `/ 16.0`. Lesson: scale inputs before you tune anything else.
- 3 **The dead layer.** ReLU hidden units started with a bias of -3. Every hidden activation is zero, the share above zero prints `0.0`, the loss sits at 0.693 and never moves. A ReLU with a negative input has slope zero, so no gradient flows back through it — ever. The fix: start biases at 0 (He initialisation for the weights). With it, 97.3%.
- 4 **The labels.** "Shuffle the training set" — applied to `X` only. Loss 0.74 to 0.70; test accuracy 54%. The network is faithfully learning images paired with random labels. The fix shuffles both with the same index: 96.9%.

THE DEBUGGING HABIT

Symptom, then hypothesis, then one print, then one change. Never change two things at once, and never change anything before printing something that could prove your hypothesis wrong. The scripts in Lab 13 are small so that the habit, not the code, is what you practise.



Why sigmoid did not die in script 3

A sigmoid with a negative input is small but its slope is not zero — gradient still trickles through. ReLU's slope is exactly zero for every negative input, which is why "dead ReLUs" are a ReLU problem specifically. Lab 13's extension A lets you confirm it.

LAB 13 · 22 MINUTES IN CLASS

Four broken training scripts

4

Lab 13 — `CSC4315_Lab13_Four_Broken_Scripts`. You have just watched the diagnosis; now do it yourself, one line changed per script, with the evidence written down: symptom, line, why the fix works.

Before Friday

1. **Finish Lab 13.**
2. **Unit 3 ends here.** Without notes: what autograd replaces, what Adam adds to SGD, why initialisation matters, and the first print for each of the four signatures.



Session 20 — Friday 13 November, 11:30

Unit 4 begins: data that fights back. A real, messy table — missing values, categories, scales — and a test score that lies because of one column.